



Airoha IoT SDK for BT Data and Smart Home Memory Layout Developer's Guide

Version: 1.0
Release date: 29 Sep 2025

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Document Revision History

Revision	Date	Description
1.0	29 Sep 2025	- Initial release. - Added AB162x memory layout description for the BT Data and Smart Home product line.

Table of Contents

Document Revision History.....	2
Table of Contents.....	3
List of Figures.....	3
List of Tables	3
1 Introduction	4
2 Memory Layout and Configuration for BT Data	5
2.1 Memory Layout and Configuration for AB162x	5
3 Memory Layout and Configuration for Smart Home	16
3.1 Memory Layout and Configuration for AB162x	16
Exhibit 1 Terms and Conditions.....	17

List of Figures

Figure 1. Serial Flash Memory Resource Brief	5
Figure 2. TCM / SYSRAM Memory Resource Brief	5
Figure 3. AB162x projects Flash Partitions with A / B Image Implementation	7
Figure 4. AB162x Projects Flash Partitions with DFU Implementation	7
Figure 5. AB162x Projects Flash Partitions with FOTA Implementation.....	8
Figure 6. Screenshot of Airoha Firmware Programming Tool	13
Figure 7. Screenshot of Airoha Flash Partitions Files	13
Figure 8. Screenshot of Memory Usage Viewed Using Visual Studio Code	15

List of Tables

Table 1. Flash Partitions and Their Meanings	6
Table 2. Terms about Execution View and Their Meaning	8
Table 3. Overview of Key Sections	9
Table 4. Introduction to Important Properties of Device Tree Binding Related to Flash Partition	12
Table 5. Special Linker Symbols to Statistics Memory Usage.....	15

1 Introduction

This document provides details on the memory layout design and configurations of Airoha IoT development for Zephyr RTOS. The chips for each product line are shown below:

- Airoha IoT SDK for BT Data: AB162x
- Airoha IoT SDK for Smart Home: AB162x

Each memory layout has two types of views: a load view and an execution view. The design concept is described based on these two views:

- The load view describes a memory region and sections of each image in terms of the address at which it is located before the image is processed.
- The execution view describes a memory region and sections of each image in terms of the address it is located during the image execution.

Different toolchains have different memory layout configuration file formats. The GCC toolchain uses a linker script. This document describes the configuration of the memory layout based on the content of a general linker script.

2 Memory Layout and Configuration for BT Data

2.1 Memory Layout and Configuration for AB162x

2.1.1 Memory Resource Overview for AB162x

The AB162x chips support three types of physical memory: Serial Flash, System Random Access Memory (SYSRAM), and Tightly Coupled Memory (TCM).

Figure 1 shows the serial flash memory part.

The AB1620D and AB1623 have 1MB of built-in flash memory, with the address ranging from 0x08000000 to 0x08100000. The AB1620DM, AB1623M, and AB1625M have 2MB of built-in flash memory, with the address ranging from 0x08000000 to 0x08200000.

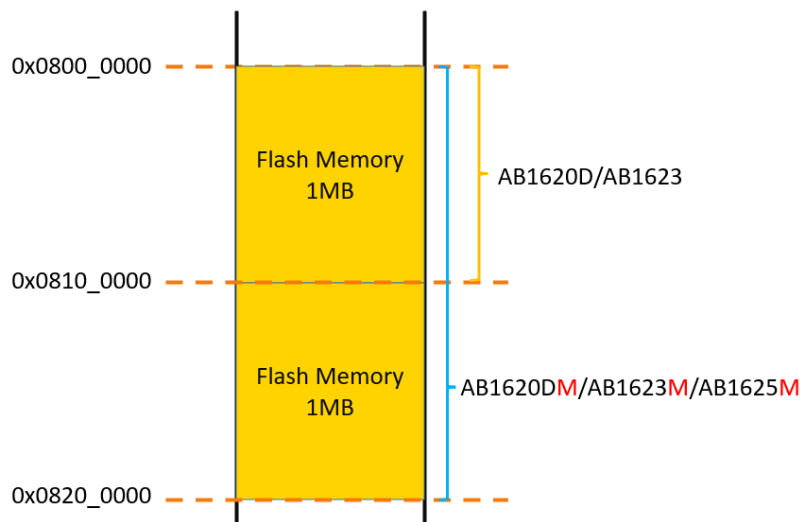


Figure 1. Serial Flash Memory Resource Brief

Figure 2 shows the RAM part.

The AB162x chips are equipped with 16KB of SYSRAM, with an address range of 0x04060000 to 0x04064000. This SYSRAM is dedicated RAM for the BT controller, designed specifically for the bus, and is generally not used for application development.

The AB162x chips are equipped with 384KB of TCM, with an address range from 0x04000000 to 0x04060000.

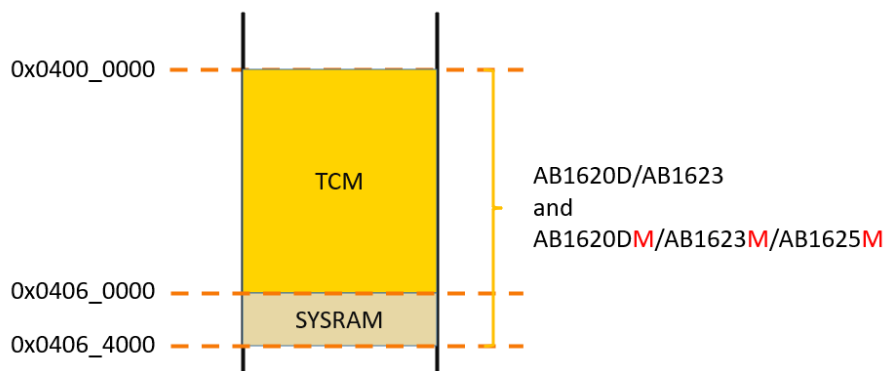


Figure 2. TCM / SYSRAM Memory Resource Brief

2.1.2 Memory Layout Programming Model for AB162x

This section introduces how to utilize the memory resources of AB162x to configure and adjust the memory layout according to the project development requirements.

This section guides you through:

- Load view / flash partitions
- Execution view
- Programming guide
- Memory layout adjustment
- Statistic memory usage
- Reserved memory mechanism

2.1.2.1 Load View / Flash Partitions

In load view, the core issue is how to allocate an appropriate flash memory size for each functional area. Due to different device update strategies, there can be significant variations in the flash partitions. The following introduces several common flash partitions.

Before discussing the flash partitions, understanding the names of certain partitions and their meanings is essential, as shown in Table 1.

Table 1. Flash Partitions and Their Meanings

Partition Name	Meaning
Partition Table	Includes the starting addresses, lengths, and other information for each partition.
Security Header	The partition required for secure boot functionality.
Factory Data	Store factory data.
MCUboot	The partition that stores MCUboot firmware allows firmware upgrades and enables transitions to the main firmware.
Upgrade Information	The partition used to store information during firmware upgrades.
MCU Firmware A/MCU Firmware	The partition where the main firmware is located.
MCU Firmware B	When selecting the upgrade strategy for the A / B image, the partition where the other main firmware is located must be the same size as the partition where MCU Firmware A is located.
Upgrade Data	The partition, which will only be available when the FOTA upgrade strategy is selected, stores the compressed firmware.
NVS	A partition for storing non-volatile data.

Figure 3 shows the default memory layout's load view of AB162x projects with A / B image. The A / B image denotes the implementation of an update strategy that requires two firmware partitions of the same size in the AB162x device to run applications.

Airoha IoT SDK for BT Data and Smart Home Memory Layout Developer’s Guide

In the 2MB memory layout, MCUboot may require support for Device Firmware Update (DFU) functionality; therefore, its partition size is 32KB larger than in a 1MB memory layout.

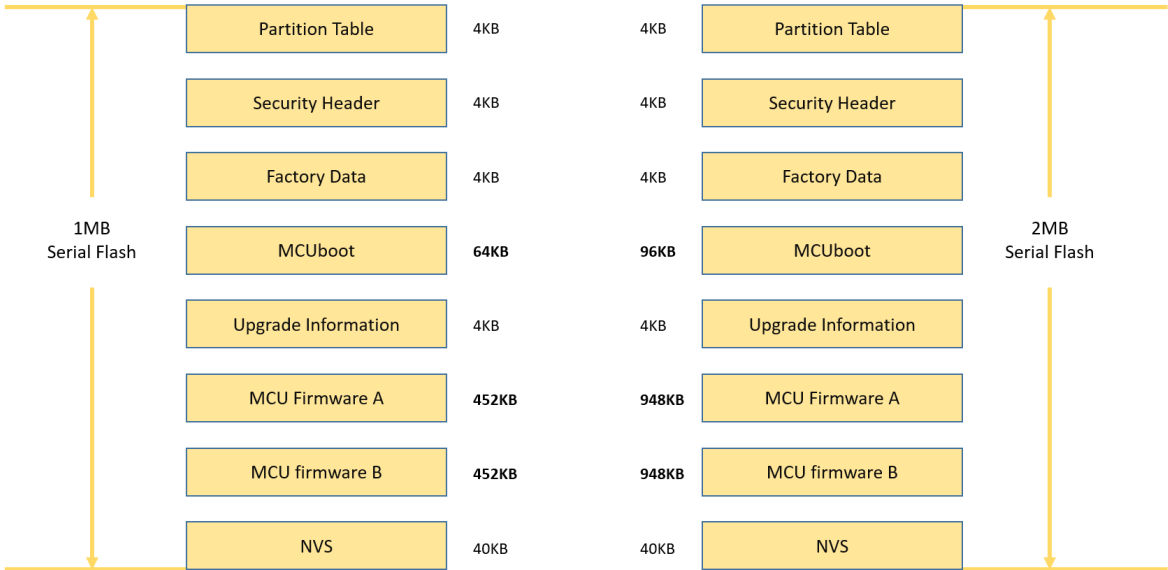


Figure 3. AB162x projects Flash Partitions with A / B Image Implementation

Figure 4 shows the default memory layout’s load view for projects that only require Device Firmware Update support.

The memory layout compatibility with DFU technology can ensure that the MCU firmware partition is as large as possible to meet the code size application requirements.

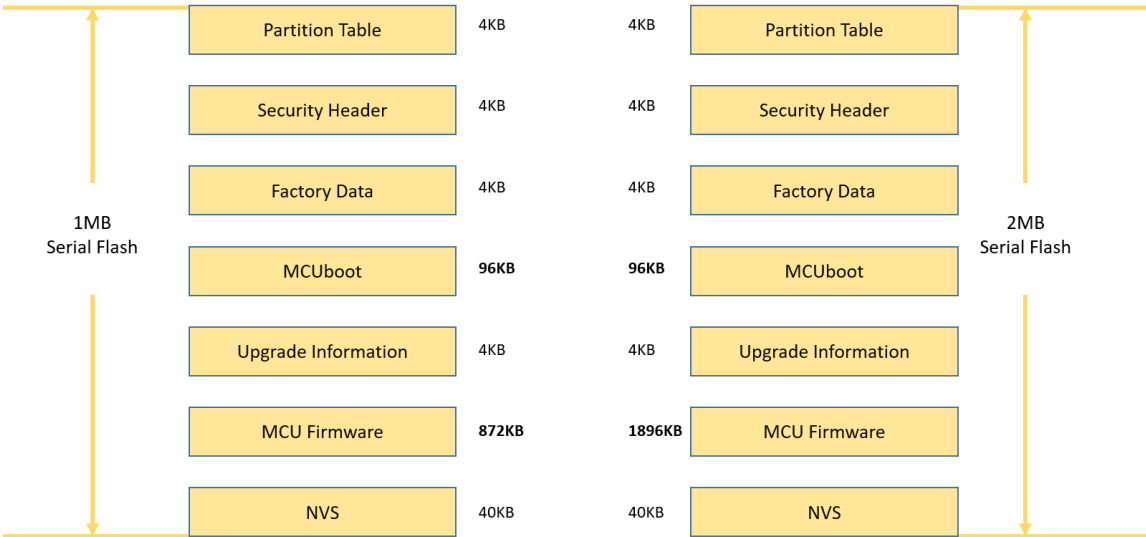


Figure 4. AB162x Projects Flash Partitions with DFU Implementation

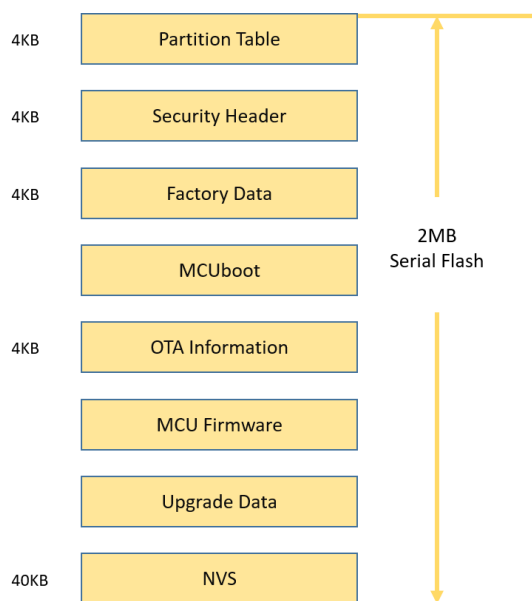


Figure 5. AB162x Projects Flash Partitions with FOTA Implementation

For requirements that need support for Firmware Over-The-Air (FOTA) and require a larger MCU firmware, Airoha offers a solution that supports the transmission of compressed firmware. Figure 5 shows the memory layout.

It should be noted that only some special partitions have a fixed size.

For partitions that do not specify a size, their size depends on the actual configuration specified in the device tree overlay file. To support different signature and decompression algorithms, the size of MCUboot may be 96 KB or 128KB.

The size of the upgrade data partition is generally 70% of the MCU firmware partition.

2.1.2.2 Execution View

The execution view refers to the actual distribution and address layout of various sections (such as the code section, data section, etc.) in the memory during firmware execution. C functions without additional attributes have identical load views and execution views. This section will not introduce them further.

Table 2 lists some commonly used terms in discussions of execution view, outlining the terms and their meanings.

Table 2. Terms about Execution View and Their Meaning

Term	Meaning
RO data	Read-only RAM data has an initial value, which is loaded from flash memory to RAM when the firmware starts executing. Generally, an MPU is used to ensure that its value is not modified. Read-only flash memory data is modified during firmware programming and cannot be directly modified like RAM at other times.
RW variable	Read-write RAM variables have an initial value, which is loaded from flash memory into RAM when the firmware starts executing.
ZI / BSS variable	Variables initialized to 0 will not occupy flash memory space.

This section describes the distribution and usage of Tightly Coupled Memory (TCM) after firmware execution. Generally used sections and their information can be referenced in Table 3, which introduces several important sections.

Table 3. Overview of Key Sections

Output Section Name	Symbols in the Map File	Meaning
.tcm_text_retention	__tcm_text_retention_start __tcm_text_retention_end	The code section is executed on the TCM.
datas	__data_start __data_end	Variables that can generally be read and written have initial values.
bss	__bss_start __bss_end	Variables that can generally be read and written without initial values.
cross_core_zi_data	_cross_core_mem_zi_data_start _cross_core_mem_zi_data_end	The ZI (Zero Initialized) data section that other cores may use.
noinit	__no_init_start __no_init_end	Data that does not require initialization to zero will be placed in this section. For example, some statically defined stack space for threads.
.tcm_text	_tcm_text_start _tcm_text_end	The code section is executed on the TCM.
.tcm_rw_bt_data_none_retention	_tcm_rw_bt_data_non_retention_start _tcm_rw_bt_data_non_retention_end	Bluetooth controller variables that can be read and written have initial values.
.tcm_rw_data_none_retention	_tcm_rw_data_non_retention_start _tcm_rw_data_non_retention_end	Other variables that can be read and written have initial values.
.tcm_zi_bt_data_none_retention	_tcm_zi_bt_data_non_retention_start _tcm_zi_bt_data_non_retention_end	Bluetooth controller variables that can be read and written with 0 as the initial value.
.tcm_zi_data_none_retention	_tcm_zi_data_non_retention_start _tcm_zi_data_non_retention_end	Other variables that can be read and written have 0 as their initial value.

The memory layout of the AB162x demonstrates specific measures taken for low power consumption: TCM code, RW variables, and Zero Initialized (ZI) variables are categorized into retention and non-retention segments. For sections with the none_retention suffix, values will not be retained during sleep. When the chip wakes up, the firmware will reinitialize these sections.

The output section consists of various segments within the final executable file or firmware image, as defined by the linker script.

It determines the layout and distribution of the program in memory. The output section allows for flexible control over the storage location and alignment of the code and data.

Examples are provided below to demonstrate how to view the output section and symbol information from the generated map file after a successful project build.

```
/* A typical example of an output section in a map file. */
/* The name of the output section is datas. */
/* The execution starting address of the output section is 0x04002de0. */
/* The length of this output section, datas, is 0x988. */
/* The load address of this output section, datas, is 0x08029938. */
/* Both __data_region_start and __data_start are at 0x04002de0. */

datas          0x04002de0  0x988 load address 0x08029938
               0x04002de0          __data_region_start = .
               0x04002de0          __data_start = .
```

2.1.2.3 Programming Guide

The programming guide is based on the content mentioned in the previous sections.

The following recommendations allow developers to successfully place the code and data in the desired memory location during runtime. The special attribute declarations listed below are located in the C header file named `memory_attribute`.

2.1.2.3.1 Put the Code or Read-Only Data into the Flash Memory

If a function block does not include specific attribute declarations, the linker places it in flash memory by default.

Variables modified by the `const` keyword are generally placed in flash memory.

```
/* example of putting code into the flash memory */
void code_into_flash_memory(void)
{ ; }

/* example of putting read-only data into the flash memory */
const int ro_data = 0xDEADBEEF;
```

2.1.2.3.2 Put the Code into the Retention TCM

Add `ATTR_TEXT_IN_TCM_RETENTION` before the function block to place the code into the retention TCM.

```
/* example of putting code into retention TCM */
#include "memory_attribute.h"
ATTR_TEXT_IN_TCM_RETENTION
void code_into_retention_tcm(void)
{ ; }
```



Note: Special attributes can only be added during function definition; they are not required during function declaration. Adding them in function declarations typically results in project build errors.

2.1.2.4 Put the Code into the None-Retention TCM

Add `ATTR_TEXT_IN_TCM` before the function block to place the code into the none-retention TCM.

```
#include "memory_attribute.h"
ATTR_TEXT_IN_TCM
void code_into_tcm(void)
{ ; }
```

2.1.2.4.1 Put Variables with Initialize Data into Retention TCM

Add ATTR_RWDATA_IN_TCM before the variable definition to place the data into the retention TCM.

```
#include "memory_attribute.h"
ATTR_RWDATA_IN_TCM
int data_with_initialize_value = 0x12345678;
```

2.1.2.4.2 Put Variables with Initialize Data into None-Retention TCM

Add ATTR_RWDATA_IN_TCM_NONE_RETENTION before the variable definition to place the data into the retention TCM.

```
#include "memory_attribute.h"
ATTR_RWDATA_IN_TCM_NONE_RETENTION
int rw_data_but_none_retention = 0x87654321;
```

2.1.2.4.3 Put Variables without Initialize Data into Retention TCM

Add ATTR_ZIDATA_IN_TCM before the variable definition to place the data into the retention TCM.

```
#include "memory_attribute.h"
/* This variable is initialized to 0 when firmware starts execution. */
ATTR_ZIDATA_IN_TCM
int zi_data;
```

2.1.2.4.4 Put Variables without Initialize Data into None-Retention TCM

Add ATTR_ZIDATA_IN_TCM_NONE_RETENTION before the variable definition to place the data into the retention TCM.

```
#include "memory_attribute.h"
/* This variable is initialized to 0 when firmware starts execution. */
ATTR_ZIDATA_IN_TCM_NONE_RETENTION
int zi_data_but_none_retention;
```



Note: The attributes of read-write data and zero-initialized data cannot be mixed. They must adhere to their respective definitions; otherwise, the behavior may not meet expectations.

2.1.2.5 Flash Partitions Adjustment

Adjusting the flash partition requires modifications to the flash partition file in device tree format and to the project settings.

2.1.2.5.1 Add Flash Partition File

Airoha referenced the native fixed-partitions structure of Zephyr RTOS and added device tree binding files for airoha,air-flash-partitions based on Airoha's development requirements. Table 4 shows the relevant information.

Table 4. Introduction to Important Properties of Device Tree Binding Related to Flash Partition

Properties	Meaning	Fixed-Partitions	Airoha,air-flash-partitions
reg	Offset and size of the flash partition relative to the base address of flash memory	Support	Support
download	Indicates whether the partition has a binary file that can be downloaded	-	Support
display_name	The name displayed on the download tool for this partition	-	Support
id	The unique ID of this partition	-	Support
customized_data	Data reserved for customers to customize	-	Support

The following example illustrates the usage of airoha,air-flash-partitions and fixed-partitions.

```
/* flash partitions in device tree output file named zephyr.dts */
flash_controller: flash-controller@40080000{
    ...
    flash0: flash@80000000 {
        ...
        main_flash_partitions: partitions {
            compatible = "airoha,air-flash-partitions", "fixed-partitions";
            ...
            boot_partition: partition@3000 {
                reg = < 0x3000 0x18000 >;
                ...
                display_name = "Bootloader";
            };
            ...
            slot0_partition: partition@1c000 {
                reg = < 0x1c000 0xed000 >;
                ...
                display_name = "MCU_FW";
            };
        };
    };
};
```

Since airoha,air-flash-partitions are compatible with fixed-partitions and extend some functionalities, both can be included in the compatible attribute of main_flash_partitions, as shown in the example above.

In the above code, two flash partitions are defined: boot_partition and slot0_partition. They are used as partitions for MCUboot and application main firmware. Their names generally cannot be modified.

The starting address of the flash memory is 0x08000000.

The boot partition has an offset of 0x3000; therefore, the starting address of this partition is 0x08003000.

Additionally, its length is 0x18000, which is equivalent to 96 KB.

The slot0 partition has an offset of 0x1c000; therefore, the starting address of this partition is 0x0801c000. Additionally, its length is 0xed000, which is equivalent to 948 KB.

With this definition, after the firmware is built, the relevant information can be recognized by the programming software for display, as shown in Figure 6.

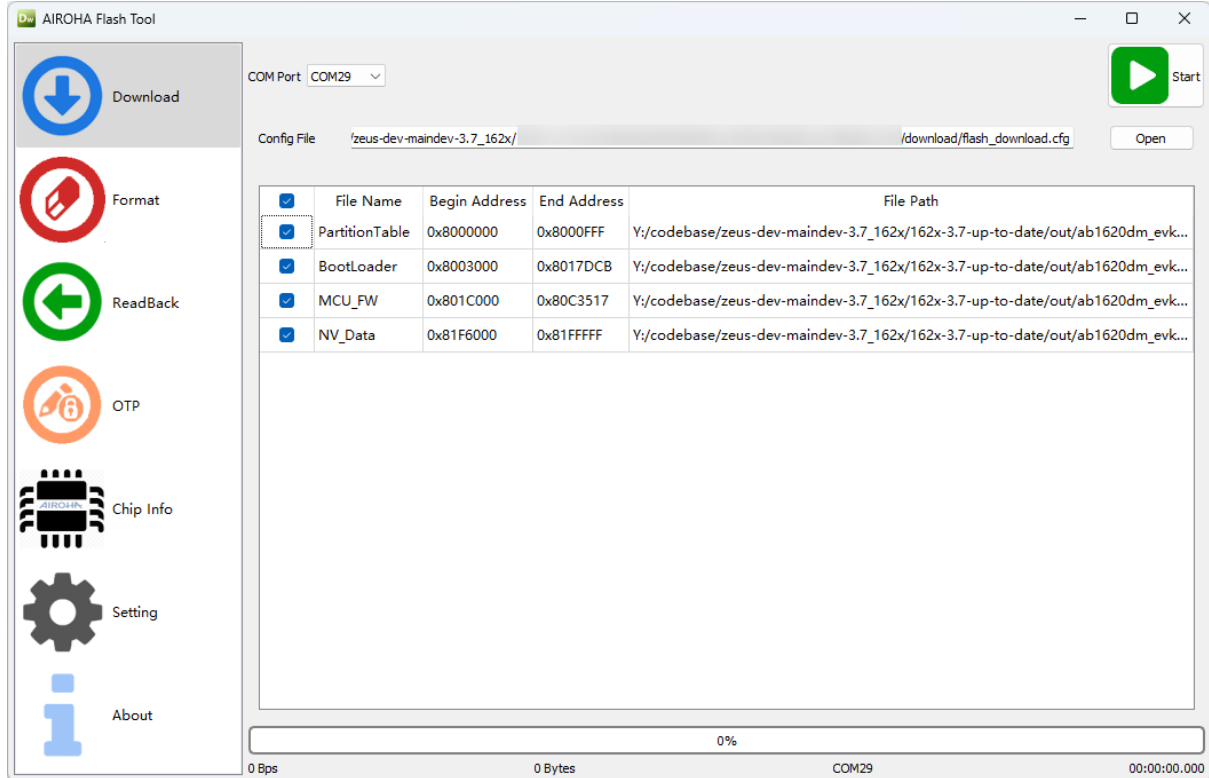


Figure 6. Screenshot of Airoha Firmware Programming Tool

With the foundational understanding established, when the flash partition needs to be modified, developers can define it by adding a new flash partition file to the **airoha/mcu/dts/arm/airoha/ab162x/flash_partitions** folder. This directory contains several examples, as shown in Figure 7. Developers can copy one example, modify its content, and rename it according to their requirements.

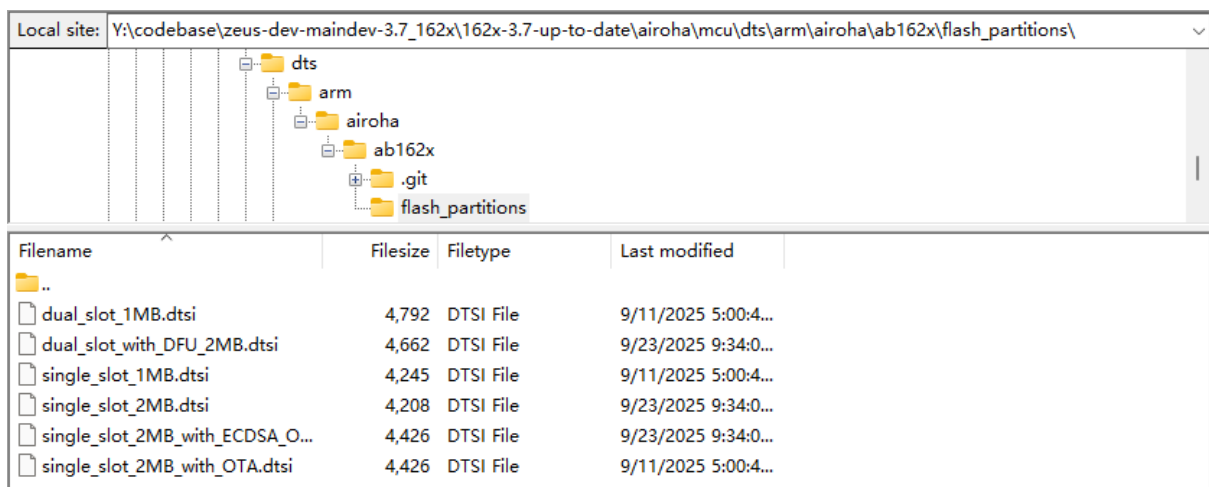


Figure 7. Screenshot of Airoha Flash Partitions Files

2.1.2.5.2 Project Settings Modification

First, developers need to have a brief understanding of the project setup. The Airoha SDK uses the YAML format to describe project configuration. The YAML format is widely used in Zephyr.

All project configurations can be found in the `project_list.yaml` file located in the `airoha/mcu/tools/mapping_proj` directory. The following code represents a typical project configuration.

```
/* build command: build.py ab1623m_evk multi_thread */
ab1623m_evk__multi_thread:
  sysbuild_conf: config/ab1623m_evk/sysbuild_with_mcuboot.conf
  proj: airoha/mcu/samples/templates/multi_thread
  mcuboot:
    # path relative to airoha/mcu/applications/mcuboot/boot/zephyr
    DTC_OVERLAY_FILE: config/ab162x_evb/mcuboot.overlay
    CONF_FILE: config/ab162x_evb/air_mcuboot.conf
  multi_thread:
    # path relative to airoha/mcu/samples/templates/multi_thread
    DTC_OVERLAY_FILE: config/ab1623m_evk/app.overlay
    CONF_FILE: config/ab1623m_evk/multi_thread.conf
```

This project consists of two parts: one involves the configuration of MCUBOOT, and the other involves the configuration of multi-threading.

The overlay is a mechanism provided by Zephyr that allows overriding certain attributes within the default device tree. Airoha specifies the overlay file by configuring the subproject's **DTC_OVERLAY_FILE**.

Zephyr uses Kconfig as the configuration system for project features. Airoha instructs Kconfig files through the subproject's **CONF_FILE**.

Adjusting the flash partition primarily involves modifying the contents of the overlay file. However, if a change in the flash partition results from a specific feature, the contents of the Kconfig configuration file must also be modified accordingly.

For example, the FOTA decompression algorithm requires the use of ECDSA. The flash partition must be replaced from `single_slot_2MB_with_OTA.dtsi` to `single_slot_2MB_with_ECDSA_OTA.dtsi`. The content of the overlay file that requires modification is shown in the following code.

```
/* Before layout modification, the mcuboot and multi_thread use the flash partitions
described at single_slot_2MB_with_OTA.dtsi */

/* content in mcuboot and multi_thread overlay file */
#include <airoha/ab162x/flash_partitions/singe_slot_2MB_with_OTA.dtsi>

/* After layout modification, the mcuboot and multi_thread use the flash partitions
described at single_slot_2MB_with_ECDSA_OTA.dtsi */

/* content in mcuboot and multi_thread overlay file */
#include <airoha/ab162x/flash_partitions/singe_slot_2MB_with_ECDSA_OTA.dtsi>
```

2.1.2.6 Statistic Memory Usage

To facilitate developers to know the amount and remaining amount of each memory, the linker script of AB162x chips include some special symbols. These special linker symbols start with `__fp_pattern_` and end with "total", "used", or "free". Below is a brief list of these linker symbols and their meanings.

Table 5. Special Linker Symbols to Statistics Memory Usage

Symbol Name	Meaning
__fp_pattern_sram_total	The total size of SRAM represented in bytes.
__fp_pattern_sram_used	The used size of SRAM represented in bytes.
__fp_pattern_sram_free	The free size of SRAM represented in bytes.
__fp_pattern_sram_bt_total	The total size of dedicated BT SRAM represented in bytes.
__fp_pattern_sram_bt_used	The used size of dedicated BT SRAM represented in bytes.
__fp_pattern_sram_bt_free	The free size of dedicated BT SRAM represented in bytes.
__fp_pattern_flash_total	The total size of current flash partition represented in bytes.
__fp_pattern_flash_used	The used size of current flash partition represented in bytes.
__fp_pattern_flash_free	The free size of current flash partition represented in bytes.

The method introduced in Section 2.1.2.2 Execution View can be used by developers to obtain information about memory resource usage from the map file.

An alternative method for viewing memory usage involves examining the contents of the build.log file located in the project output folder. This method is straightforward and easy to understand. The following figure illustrates an example of viewing the memory usage after building a multi-thread project.

```

out > ab1623m_evk > multi_thread > build.log
802 [321/321] Linking CXX executable zephyr/zephyr.elf
803 Memory region      Used Size  Region Size  %age Used
804 |-----| FLASH:    141900 B    948 KB      14.62%
805 |-----| SRAM:      88208 B    384 KB      22.43%
806 |-----| SRAM_BT:    0 GB      16 KB       0.00%
807 |-----| ROM_LOG_STRING: 61641 B    32 MB       0.18%
808 |-----| IDT_LIST:    0 GB      2 KB       0.00%

```

Figure 8. Screenshot of Memory Usage Viewed Using Visual Studio Code

3 Memory Layout and Configuration for Smart Home

3.1 Memory Layout and Configuration for AB162x

For the smart home product line, the configuration method of the AB162x memory layout is identical to that of Bluetooth (BT) Data. Refer to the description in Section 2.1 Memory Layout and Configuration for AB162x.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this "Document") is subject to your (including the corporation or other legal entity you represent, collectively "You") acceptance of the terms and conditions set forth below ("T&C"). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don't agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to Airoha Technology Corp. and/or its affiliates (collectively "Airoha") or its licensors and is provided solely for Your internal use with Airoha's chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against Airoha or any of Airoha's suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify Airoha for any loss or damages suffered by Airoha for Your unauthorized use or disclosure of this Document, in whole or in part.

Airoha and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. Airoha does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY AIROHA IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. AIROHA SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. AIROHA SHALL NOT BE RESPONSIBLE FOR ANY AIROHA DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, Airoha makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does Airoha assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating Airoha's product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.